

Introduction

This is the documentation of [spotifyd](#), which covers the installation and setup process as well as some advanced topics.

Getting Help

If you're stuck in the setup process or have a question, please join the [community matrix server](#). We're always willing to help!

Installation

Getting `spotifyd` on your system should be as easy as downloading a binary in most cases. If you'd like to learn how to compile `spotifyd` yourself, head over to [building from source](#).

Linux

Some linux distributions include `spotifyd` in their official repositories. Have a look at [Repology](#) for a list of the distributions that currently ship an up-to-date version of `spotifyd`.

If your distribution is not supported or the provided version is too old, skip to [this section](#) in order to install one of our pre-built binaries.

macOS

If you're a homebrew user, installing `spotifyd` is as easy as running

```
brew install spotifyd
```

FreeBSD

On FreeBSD, a package is available and can be installed with `pkg install spotifyd`.

OpenBSD

On OpenBSD, a package is available and can be installed with `pkg_add spotifyd`.

Installing from releases

If none of the above methods work for you, you can also use our provided binaries.

First, you need to find a suitable binary for your platform. The provided binaries differ in the available features and the platform architecture that they were built for. You can find the latest binaries [here](#).

Feature Sets:

- `full`: **all audio backends** and **MPRIS** support
- `default`: **some audio backends** (depending on your platform: PulseAudio, PortAudio, ALSA) and **MPRIS** support
- `slim`: **one audio backend** (depending on your platform) and **no MPRIS** support (good for headless systems)

If you're unsure which version to choose, just go for `default` on desktop systems and `slim` on headless systems.

Architecture:

If you're on Linux, check your platform architecture with `uname -m`:

- `x86_64`: Download one of the `spotifyd-linux-x86_64-{full,default,slim}.tar.gz` packages.
- `armhf`, `armv7`: Download one of the `spotifyd-linux-armv7-{full,default,slim}.tar.gz` packages.
- `aarch64`: Download one of the `spotifyd-linux-aarch64-{full,default,slim}.tar.gz`
- `armv6`: Unfortunately, we no longer support this architecture. If you still need this to work, please open an issue or join the [community matrix channel](#) and we'll try to find a solution.

If you're on macOS, download one of the `spotifyd-macos-{full,default,slim}.tar.gz` packages.

You should now extract the downloaded archive, make the `spotifyd` file executable and copy it to a sensible location. This can be done using the following commands:

```
$ tar xzf spotifyd-*.tar.gz # extract
$ cd spotifyd-*/
$ chmod +x spotifyd # make binary executable
$ # move to correct location, e.g. on Linux:
$ # for a user-wide installation (make sure that your $PATH includes
~/local/bin)
$ mv spotifyd ~/.local/bin/spotifyd
$ # for a system-wide installation
$ sudo chown root:root spotifyd
$ sudo mv spotifyd /usr/local/bin/spotifyd
```

Running

Now that you have installed `spotifyd`, you can check if everything was successful by running `spotifyd --version`.

You should be ready to go now and after running `spotifyd --no-daemon`, it should appear in an **official** Spotify client which is on the same network. If this does not work, you can head over to the [troubleshooting section](#) or look at [different methods of authentication](#).

Running as a service

Most people want to have `spotifyd` always running in the background. The preferred method to do this depends on the OS you're using.

Linux

If you installed `spotifyd` directly from your distribution, chances are that the installation includes a service definition such that you can run

```
$ systemctl --user start spotifyd # start spotifyd using systemd
$ systemctl --user enable --now spotifyd # start spotifyd and enable starting
on login
```

If you installed `spotifyd` some other way, head over to [the advanced section](#) for further instructions.

macOS

If you installed `spotifyd` using brew, the following commands should do the trick

```
$ brew services run spotifyd # start the service once
$ brew services start spotifyd # start spotifyd and enable starting on boot
```

If you installed `spotifyd` without brew, [the advanced section](#) has got you covered.

FreeBSD

When installed via the package manager, the following commands are available:

```
$ sudo service spotifyd onestart # start spotifyd once
$ sudo sysrc spotifyd_enable=YES # enable starting spotifyd on boot
```

Building from source

The guide below assumes that you're building `spotifyd` on the system that you want to run it on. If you'd instead prefer to cross-compile, head over to [this section](#).

You can also compile `spotifyd` yourself, allowing you to tailor it perfectly to your needs or get the latest fixes. `spotifyd` is written in Rust. You can download the toolchain (compiler and package manager) over at rustup.rs. Follow their instructions to get started.

Note: Please make sure that you compile the package using the most recent `stable` version of Rust available through `rustup`. Some distro versions are quite outdated and might result in compilation errors.

Required packages

`spotifyd` might require additional libraries during build and runtime, depending on your platform and the way to compile it (static or dynamic). The following table shows the libraries needed for each OS respectively.

Target Platform	Libraries
Fedora	alsa-lib-devel make gcc
openSUSE	alsa-devel make gcc
Debian	libasound2-dev libssl-dev libpulse-dev libdbus-1-dev
Arch	base-devel alsa-lib libogg libpulse dbus
macOS	dbus pkg-config portaudio

If you're building on a non-standard target, for example the RaspberryPi, you might need an additional `libclang-dev` and `cmake` package for one of our dependencies. Details can be found on [this page](#).

Installing with cargo

To build and install the latest version of `spotifyd`, you can use the package manager for rust. The base command is the following

```
cargo install spotifyd --locked
```

If you would rather install from the latest commit on GitHub, download or clone the project, enter the directory and run

```
cargo install --path . --locked
```

Note: Both methods will install the binary in `$XDG_DATA_HOME/cargo/bin` or `$HOME/.cargo/bin`. To execute it, you must make sure that this location is part of your shell's `$PATH` variable. Also, you might have to change the paths when following other parts of this documentation.

Compiling with cargo

To just build the binary without installing, run

```
cargo build --release --locked
```

This will build the binary and leave the result at `./target/release/spotifyd`.

Building a Debian package

You can use the `cargo-deb` crate in order to build a Debian package from source. Install it by:

```
cargo install cargo-deb
```

Then you can build and install the Debian package with:

```
cargo deb --install
```

Note, that when building a Debian package, the `--release` is passed to the build command already and you do not need to specify it yourself. See for the flags that are set by default in `Cargo.toml`.

Building an RPM package

You can use the `cargo-generate-rpm` crate in order to build an RPM package from source. Install it by:

```
cargo install cargo-generate-rpm
```

Then you can build and install the RPM package with:

```
cargo build --release
strip -s target/release/spotifyd
cargo generate-rpm
sudo rpm -Uvh target/generate-rpm/spotifyd-[VERSION].[ARCH].rpm
```

Feature Flags

`spotifyd` is split into a base package plus additional features that can be toggled on or off during compilation. Those can be split into two groups: The audio backend features that are responsible for playing back the music and additional functionality features, which enhance your experience using `spotifyd`.

Feature Flag	Description
<code>alsa_backend</code>	Provides support for the ALSA backend. Should work in most setups and is enabled by default.
<code>pulseaudio_backend</code>	Support for PulseAudio.
<code>rodio_backend</code>	Rust-native implementation of audio backends on all platforms. Does not need any external packages to be installed.
<code>portaudio_backend</code>	Audio backend that can be used on non-Linux systems.
<code>rodiojack_backend</code>	Support for the Jack backend.
<code>dbus_mpris</code>	Provides multimedia key support (Linux and BSD only)

To customize your build, pass a subset of the features listed above to any of the `cargo` commands above via `--features <feature1>,<feature2>,...`. Disable the default feature `alsa_backend` with `--no-default-features`. So an example command could look like the following:

```
cargo install spotifyd --locked --no-default-features --features
rodio_backend,dbus_mpris
```

Cross-Compilation

If you want to run `spotifyd` on lower-power hardware such as a RaspberryPi, but none of our prebuilt binaries suit your needs, you might want to cross-compile `spotifyd` on a more powerful machine and deploy the binary on the target system.

Using cross

The easiest way to cross-compile is using the amazing [cross project](#). This way, the build environment comes already pre-configured. Follow the instructions in their README to install `cross`.

 In the current latest release of `cross` (v0.2.5), some targets are too outdated and compilation will fail. Thus, it is currently recommended to install the latest version of `cross` from their git repo instead of a prebuilt binary.

Then, you should be able to run `cross build --target <your desired target>`, where `target` is one of the targets in `rustc --print target-list`. Please also refer to [the general from source guide](#) for additional flags that you might want to append to that command.

If this was successful, copy the resulting binary from `target/<your desired target>/{release,debug}/spotifyd` to the `spotifyd` machine and try running it there.

If `cross` doesn't support your target, you can try the alternative approach using Docker and QEMU below.

If compilation of your target isn't working even though `cross` supports it, feel free to open an issue on our GitHub or join the [community matrix channel](#) and ask there.

Using Docker and QEMU

We can also use `docker` to cross compile on every platform and OS that runs `docker` and `qemu`:

1. Setup a docker [custom builder](#)

```
docker buildx create \
  --name container-builder \
  --driver docker-container \
  --use --bootstrap
```

If you are **not** using Docker-Desktop you might have to install [QEMU](#)

2. Create a docker `docker-compose.yml`

Here we are building a `arm64` binary, so we set `platform: linux/arm64`

```
services:
  build-container:
    image: rust:1-bookworm
    platform: linux/arm64
    command: bash -c "
      apt-get update &&
      apt-get install -y \
        libasound2-dev \
        libssl-dev \
        jq \
        pkg-config &&
      wget -O -
https://api.github.com/repos/Spotifyd/spotifyd/tarball/$(\
      curl -SsL
https://api.github.com/repos/Spotifyd/spotifyd/releases/latest \
        | jq '.tag_name' -r) \
        | tar xzv -C /spotifyd --strip-components=1 &&
      cargo build --release &&
      cp /spotifyd/target/release/spotifyd /build/"
    working_dir: /spotifyd
    volumes:
      - .:/build
```

3. Run `docker compose up`

This will copy the build `spotifyd` binary in the current directory.

Configuration

`spotifyd` is able to run without configuration at all and will assume default values for most of the fields. However, running without configuration will only allow you to connect to it via Spotify Connect if you're on the same network as the daemon.

All configuration options can be specified directly as command line arguments or alternatively (in particular for permanent changes) be written into a configuration file. Throughout this section, we will always give both possibilities, e.g. `--cli-arg` for the cli variant / `config_value` for the config file variable.

Config File

`spotifyd` will look for its configuration at `/etc/spotifyd.conf`, `$XDG_CONFIG_PATH/spotifyd/spotifyd.conf` or if that is not set `~/.config/spotifyd/spotifyd.conf`. For other needs, you can point `spotifyd` to it's config file with the command line argument `--config-path <path-to-your-config>`.

You can start with the following documented config as an example and read through the subpages of this section:

[global]

```
#-----#
# GENERAL #
#-----#

# The name that gets displayed under the connect tab on
# official clients.
#device_name = "device_name_in_spotify_connect"

# The displayed device type in Spotify clients.
# Can be unknown, computer, tablet, smartphone, speaker, t_v,
# a_v_r (Audio/Video Receiver), s_t_b (Set-Top Box), and audio_dongle.
#device_type = "speaker"

# The directory used to store credentials and audio cache.
# Default: infers a sensible cache directory (e.g. on Linux: $XDG_CACHE_HOME)
#
# Note: The file path does not get expanded. Environment variables and
# shell placeholders like $HOME or ~ don't work!
#cache_path = "/full/path/to/cache/directory"

# If set to true, audio data does NOT get cached.
# In this case, the cache is only used for credentials.
#no_audio_cache = true

# The maximal size of the cache directory in bytes
# The example value corresponds to ~ 1GB
#max_cache_size = 1000000000

# If set to true, `spotifyd` tries to bind to dbus (default is the session bus)
# and expose MPRIS controls. When running headless, without the session bus,
# you should set this to false, to avoid errors. If you still want to use
# MPRIS,
# have a look at the `dbus_type` option.
#use_mpris = true

# The bus to bind to with the MPRIS interface.
# Possible values: "session", "system"
# The system bus can be used if no graphical session is available
# (e.g. on headless systems) but you still want to be able to use MPRIS.
# NOTE: You might need to add appropriate policies to allow spotifyd to
# own the name.
#dbus_type = "session"

#-----#
# DISCOVERY #
#-----#

# If set to true, this disables zeroconf discovery.
# This can be useful, if one prefers to run a single-user instance.
#disable_discovery = false

# The port at which `spotifyd` is going to offer its service over the network
# (TCP).
# If not set, a random port > 1024 is used. For the service to be discoverable
# on the
```

```

# local network via mDNS, both the mDNS port (5353 UDP) and the random or fixed
# zeroconf port need to be allowed through any active firewall.
#zeroconf_port = 1234

#-----#
# AUDIO #
#-----#

# The audio backend used to play music. To get
# a list of possible backends, run `spotifyd --help`.
#backend = "alsa" # use portaudio for macOS [homebrew]

# The alsa audio device to stream audio. To get a
# list of valid devices, run `aplay -L`,
#device = "default" # omit for macOS

# The PCM sample format to use. Possible values
# are F32, S32, S24, S24_3, S16.
# Change this value if you encounter errors like
# "Alsa error PCM open ALSA function 'snd_pcm_hw_params_set_format' failed with
error 'EINVAL: Invalid argument'"
#audio_format = "S16"

# The volume controller. Each one behaves different to
# volume increases. For possible values, run
# `spotifyd --help`.
#volume_controller = "softvol" # use softvol for macOS

# ! Only relevant for ALSA !
# The alsa control device. By default this is the same
# name as the `device` field.
#control = "default"

# ! Only relevant for ALSA !
# The alsa mixer used by `spotifyd`.
#mixer = "PCM" # omit for macOS

# The audio bitrate. 96, 160 or 320 kbit/s
#bitrate = 160

# Volume on startup between 0 and 100
#initial_volume = 90

# If set to true, enables volume normalisation between songs.
#volume_normalisation = true

# The normalisation pregain that is applied for each song.
#normalisation_pregain = -10

#-----ä
# OTHER #
#-----#

# After the music playback has ended, start playing similar songs based on the
previous tracks.
# By default, `spotifyd` infers this setting from the user settings.
#autoplay = true

```

```
# A command that gets executed in your shell after each song changes.  
#on_song_change_hook = "echo \"hook executed on $PLAYER_EVENT\""  
  
# The proxy `spotifyd` will use to connect to spotify.  
#proxy = "http://proxy.example.org:8080"
```

Authentication

There are two different ways of authentication supported.

Discovery on LAN

By default, `spotifyd` advertises itself on the local network as a Spotify Connect device and will thus appear in **official** clients. After selecting it over there, you should be able to hear the sound coming out of your `spotifyd` device.

For this to work, you need to make sure that your firewall isn't blocking the discovery. In particular, `spotifyd` uses two ports:

- `5353 UDP` : MDNS service advertisement
- A zeroconf port which uses TCP. By default, it is randomly chosen, but if you want to, you can configure it with the `--zeroconf-port` cli option / `zeroconf_port` config value.

If you don't want discovery, because you're using one of the methods below, you can disable it via the `--disable-discovery` cli option / `disable_discovery = true` config value.

Note: By default, the last active session will be remembered and reconnected once the service is restarted.

Manual Login (via OAuth)

If for some reason, discovery is not a viable option for your use case or you prefer a single-user instance, you can manually log in to your account and `spotifyd` will connect to this account by default.

Note: Since the login method requires a web browser, trying this on headless systems is not recommended. As a workaround, you can log in on a machine with a working web browser and then copy the credential file onto the headless system. The location of the credential file will be `<cache_path>/oauth/credentials.json`.

Before you begin the login flow, make sure that you won't need to change `cache_path` later on, because that location will be used to store the login data.

Now, you're ready to run `spotifyd authenticate` (for available options, see `spotifyd auth --help`). This will ask you to browse to a link with your preferred web browser. On the page, you need to log into your Spotify account and confirm the connection.

If the process was successful, you should see the message "Go back to your terminal :)" in your browser window. You can now close the tab and return to the terminal.

Now, when running `spotifyd --no-daemon`, you should see that `spotifyd` automatically connects to your account.

```
Loading config from "..."  
[...]  
Login via OAuth as user <your username>.  
[...]  
Authenticated as '<your username>' !
```

Note: Even if you logged into `spotifyd` using this method, discovery will still be enabled by default and any incoming connection will interrupt the current session. If you don't want or need this, you can disable it via the `--disable-discovery` cli option / `disable_discovery = true` config value.

Audio Configuration

On most setups, audio should be Just Working™. If you have specific needs or something's not working, you might need to touch some of the config values.

Backend

`-b/--backend` or `backend` in the config file.

There are different audio backends available to choose from. See `spotifyd --help` to get the available names.

Device selection

`--device` or `device` in the config file.

Instead of using the default device, which can sometimes be different to what you'd prefer, you can ask `spotifyd` to use a specific audio device. The interpretation and available values depends on the backend you're using.

- For ALSA, you can use `aplay -L` to get a list of possible devices.
- For PulseAudio, run `pactl list short sinks` to get a list of possible names.

Bitrate

`-B/--bitrate` or `bitrate` in the config file.

To reduce bandwidth usage or increase quality, you can play with the bitrate.

Volume Controller

`--volume-controller` or `volume_controller` in the config file.

In most cases, leaving this at the default (`softvol`) should be fine.

If you want your `spotifyd` volume to be synchronized with an output device's hardware volume, you can set this to `alsa` or `alsa_linear` . In both cases, you might also want to set the `mixer` device to set which device's volume should be changed.

If you want to prevent the user to be able to adjust the volume, set this instead to `none` .

Other

For more interesting but less relevant audio options, have a look at `spotifyd --help` or [the example config](#).

Other settings

Appearance

You can customize, how your device is displayed in clients with `--device-name / device_name` and `--device-type / device_type` options.

Caching

You can enable audio data caching by setting a `--cache-path / cache_path` and limit its size with `--max-cache-size / max_cache_size`.

MPRIS

On linux desktop systems, you can enable `--use-mpris / use_mpris` (if your version has enabled that feature). This will give your desktop environment or tools like `playerctl` the option to display information about and control `spotifyd`.

Advanced Setup

In this section, you will learn how to persist `spotifyd` as a system service, run hook scripts on certain events and control `spotifyd` via DBUS on headless systems.

Running as systemd service

As as a user service

A `systemd.service` unit file is provided in the project sources to help run `spotifyd` as a service on systemd-based systems. The file `contrib/spotifyd.service` should be copied to either:

```
/etc/systemd/user/  
~/.config/systemd/user/
```

Packagers of systemd-based distributions are encouraged to include the file in the former location. End-user should prefer the latter. It should be noted that some targets are not available when running under the user directory, such as `network-online.target`.

Control of the daemon is handed over to systemd. The following command will start the service whenever the user logs in to the system. Logging out will stop the service.

```
systemctl --user enable spotifyd.service --now
```

As a system wide service

 When running `spotifyd` as a system wide service, it is not possible to access a user's keyring to obtain login credentials. Do not set `use_keyring = true` and do not specify `--use-keyring`, when running as a system wide service. To be able to access login credentials stored in the user's keyring, run `spotifyd` as a user service, as described above.

Additionally, `use_mpris = true` or `--use-mpris` should not be used, since their intended usage is within user sessions (and not system-wide daemons). If you have very specific requirements and still want to control a system-wide `spotifyd` instance, head over to [MPRIS on headless systems](#).

A `systemd.service` unit file is provided to help run `spotifyd` as a service on systemd-based systems. The file `contrib/spotifyd.service` should be copied to:

```
/etc/systemd/system/
```

Control of the daemon is handed over to systemd. The following example commands will start the service and keep it running across reboots.

```
systemctl daemon-reload  
systemctl enable spotifyd.service --now
```

Running as launchd service

On macOS, the system wide and per-user daemon/agent manager is known as `launchd`. Interfacing with `launchd` is performed through `launchctl`.

In order to use `spotifyd` as a service on macOS one must specify a `.plist` that represents the service, and place it in `/Library/LaunchDaemons`.

Here is a `.plist` which works with macOS Catalina 10.15.3:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <dict>
    <key>Label</key>
    <string>rustlang.spotifyd</string>
    <key>ProgramArguments</key>
    <array>
      <string>/usr/local/bin/spotifyd</string>
      <string>--config-
path=/users/YourUserName/.config/spotifyd/spotifyd.conf</string>
      <string>--no-daemon</string>
    </array>
    <key>UserName</key>
    <string>YourUserName</string>
    <key>KeepAlive</key>
    <true/>
    <key>ThrottleInterval</key>
    <integer>30</integer>
  </dict>
</plist>
```

Once present in the `/Library/LaunchDaemons` directory, the `.plist` must be loaded and started with the following commands.

```
sudo launchctl load -w /Library/LaunchDaemons/rustlang.spotifyd.plist
```

```
sudo launchctl start /Library/LaunchDaemons/rustlang.spotifyd.plist
```

One may also unload/stop the service in a similar fashion replacing load/start with unload/stop.

Note:

- You should update "YourUserName" with your actual username for macOS (or remove "UserName" to run as root).
- The string, `<string>--no-daemon</string>` is needed as `launchd` won't receive a PID for the process and will lose its remit over `spotifyd`. So it's best to include it, there will

be no difference in use, nor will you see any log output.

- macOS tries to start the daemon immediately on boot, and spotifyd fails if Wifi isn't connected. So one must have a keep alive (which retries if it fails to launch on boot), that retries after 30 seconds, which is enough for wifi etc to come up.

Using D-Bus to control spotifyd

If MPRIS support is built into your version and enabled (`--use-mpris` cli flag / `use_mpris = true` in config), `spotifyd` exposes some interfaces via D-Bus through which it provides information and can be controlled.

Most of the time, you won't have to worry too much about the details, since tools like `playerctl` work out of the box with `spotifyd`. If you have some custom requirements or want to write custom scripts to control `spotifyd`, this section is for you.

Available Interfaces

Directly after startup, no interfaces will be available. Once we are connected to Spotify, `spotifyd` will request the name `rs.spotifyd.instance$PID` (where `PID=$(pidof spotifyd)`) and expose the interface `rs.spotifyd.Controls`.

As soon as we are the playback device (e.g. because we are selected from another client or the `TransferPlayback` method has been called), `spotifyd` will additionally expose the MPRIS interfaces and request the name `org.mpris.MediaPlayer2.spotifyd.instance$PID`.

Spotifyd Controls

The `rs.spotifyd.Controls` interface exposes a few useful controls that are available even if we're not the active playback device.

- Method `TransferPlayback`: transfers Spotify playback to `spotifyd`
- Method `VolumeUp`: increases player volume
- Method `VolumeDown`: decreases player volume

Examples:

```
dest=rs.spotifyd.instance$(pidof spotifyd)
# increase volume
dbus-send --print-reply --dest=$dest /rs/spotifyd/Controls
rs.spotifyd.Controls.VolumeUp
# become the active playback device
dbus-send --print-reply --dest=$dest /rs/spotifyd/Controls
rs.spotifyd.Controls.TransferPlayback
```

MPRIS

The `org.mpris.MediaPlayer2` and `org.mpris.MediaPlayer2.Player` interfaces from the [MPRIS specification](#) are implemented.

Example usage:

```
dest=org.mpris.MediaPlayer2.spotifyd.instance$(pidof spotifyd)
# Start playback of some Spotify URI
dbus-send --print-reply --dest=$dest /org/mpris/MediaPlayer2
org.mpris.MediaPlayer2.Player.OpenUri
string:spotify:track:4PTG3Z6ehGkBFwjybzWkR8
# Get metadata of the currently playing track
dbus-send --print-reply --dest=$dest /org/mpris/MediaPlayer2
org.freedesktop.DBus.Properties.Get string:org.mpris.MediaPlayer2.Player
string:Metadata
```

Examples

Starting Playback without Client:

```
#!/bin/bash

# optionally, we can start `spotifyd` here
if ! pidof -q spotifyd
then
    spotifyd --use-mpris
fi

dest=rs.spotifyd.instance$(pidof spotifyd)

wait_for_name() {
    dst=$1
    counter=0

    # check if controls are available
    until [ $counter -gt 10 ] || (dbus-send --print-reply --
dest=org.freedesktop.DBus /org/freedesktop/DBus org.freedesktop.DBus.ListNames
| grep -q "$dest")
    do
        sleep 0.3
        ((counter++))
    done

    if [ $counter -gt 10 ]
    then
        echo "waiting for spotifyd timed out" >&1
        exit 1
    fi
}

controls_name=rs.spotifyd.instance$(pidof spotifyd)
wait_for_name $controls_name
echo "Transferring Playback"
dbus-send --print-reply --dest=$controls_name /rs/spotifyd/Controls
rs.spotifyd.Controls.TransferPlayback

# if URI is specified, start the playback there
if [ -n "$1" ]
then
    uri="$1"
    mpris_name=org.mpris.MediaPlayer2.spotifyd.instance$(pidof spotifyd)
    wait_for_name $mpris_name
    echo "Starting Playback of $uri"
    dbus-send --print-reply --dest=$mpris_name /org/mpris/MediaPlayer2
org.mpris.MediaPlayer2.Player.OpenUri "string:$uri"
else
    echo "Hint: specify an argument to start playback of a specific Spotify URI"
fi
```

Sleep Timer:

```
#!/bin/bash

usage() {
    echo "Usage: $0 <timeout>" >&1
    exit 1
}

[ -n "$1" ] || usage

echo "Sleeping for $1 seconds"
sleep $1

dest=org.mpris.MediaPlayer2.spotifyd.instance$(pidof spotifyd)
dbus-send --print-reply --dest=org.freedesktop.DBus /org/freedesktop/DBus
org.freedesktop.DBus.ListNames | grep -q "$dest"

if [ "$?" = "0" ]
then
    dbus-send --print-reply --dest=$dest /org/mpris/MediaPlayer2
org.mpris.MediaPlayer2.Player.Stop
    # alternatively just pause:
    # dbus-send --print-reply --dest=$dest /org/mpris/MediaPlayer2
org.mpris.MediaPlayer2.Player.Pause
else
    echo "No active spotifyd playback."
fi
```

MPRIS on headless systems

D-Bus offers two types of communication buses, the system and the session bus. By default and in most setups, MPRIS is expected to run on the session bus (for example, `playerctl` requires that). However, in headless contexts, the session bus is usually not available.

In this case, you basically have two options:

Option 1: Launch with `dbus-launch`

By creating a wrapper script and using `dbus-launch`, `spotifyd` can be run with its own bus. This could look like the following

```
./spotify_wrapper.sh:  
  
#!/bin/bash  
  
echo "$DBUS_SESSION_BUS_ADDRESS" > /tmp/spotifyd_bus  
echo "To use spotifyd's session bus, run 'export DBUS_SESSION_BUS_ADDRESS=$(cat  
/tmp/spotifyd_bus)'"  
spotifyd --no-daemon --use-mpris
```

Then, execute this script using `dbus-launch ./spotify_wrapper.sh` and follow the instructions in the output.

Option 2: Using the system bus

Instead of creating a new session bus, we can instead make use of the always existing system bus. To instruct `spotifyd` to use the system instead of the session bus, set the `--dbus-type system` cli flag / `dbus_type = "system"` config value.

By default, requesting names on the system bus requires special privileges, which `spotifyd` doesn't have. So, unless being run as root, this will fail. To allow a non-root user to request the `spotifyd` name, we need to create the following file (replacing `your` user by something sensible):

```
/usr/share/dbus-1/system.d/spotifyd.conf:
```

```
<!DOCTYPE busconfig PUBLIC
    "-//freedesktop//DTD D-BUS Bus Configuration 1.0//EN"
    "http://www.freedesktop.org/standards/dbus/1.0/busconfig.dtd">
<busconfig>
  <!-- Only this user can own the spotifyd interfaces -->
  <policy user="your user">
    <allow own_prefix="rs.spotifyd"/>
    <allow own_prefix="org.mpris.MediaPlayer2.spotifyd"/>
  </policy>

  <!-- Allow this user, to invoke methods on these two interfaces -->
  <policy user="your user">
    <allow send_destination_prefix="rs.spotifyd"/>
    <allow send_destination_prefix="org.mpris.MediaPlayer2.spotifyd"/>
  </policy>
</busconfig>
```

Make sure to reload the D-Bus configuration with `systemctl reload dbus`.

User supplied Scripts

If you want to create custom behaviour around `spotifyd`, one option is to look into the [D-Bus functionality](#). But on headless systems or smaller projects, we also offer a hook API. On certain events, a hook can be executed with event details given as environment variables.

To point `spotifyd` to such a script, use the `--onevent /path/to/script cli arg / on_song_change_hook = "/path/to/script"` configuration value.

In order to learn about the available events and the available details, you can either create simple scripts which log the given environment variables or look at the output of `spotifyd`, which logs whenever the script is executed.

The following scripts are intended to serve as inspiration for your own scripts. If you have written own scripts which you think might be useful to others, please create a PR adding them here!

Dunst Notifications (Using Spotify API)

This script will show a dunst notification when you play/change/stop Spotify (and when the music change). It is using spotify APIs to get music details.

Dependencies

- curl (Request to APIs)
- xargs (Argument passing)
- cut
- jq (<https://stedolan.github.io/jq>) (for JSON parsing)

How to use

- Create a file containing the script below:

```
user_id=YOUR_USER_ID # generated on
https://developer.spotify.com/dashboard/applications
secret_id=YOUR_SECRET_ID
```

```
myToken=$(curl -s -X 'POST' -u $user_id:$secret_id -d
grant_type=client_credentials https://accounts.spotify.com/api/token | jq
'.access_token' | cut -d\" -f2)
RESULT=$?
```

```
if [ "$PLAYER_EVENT" = "start" ];
then
    if [ $RESULT -eq 0 ]; then
        curl -s -X 'GET' https://api.spotify.com/v1/tracks/$TRACK_ID -H
'Accept: application/json' -H 'Content-Type: application/json' -H
"Authorization: \"Bearer $myToken\"" | jq '.name, .artists[].name,
.album.name, .album.release_date, .track_number, .album.total_tracks' |
xargs printf "\"Playing '%s' from '%s' (album: '%s' in %s (%s/%s))\"" |
xargs notify-send --urgency=low --expire-time=3000 --
icon=/usr/share/icons/gnome/32x32/actions/player_play.png --app-
name=spotifyd spotifyd
    else
        echo "Cannot get token."
    fi
elif [ "$PLAYER_EVENT" = "change" ];
then
    if [ $RESULT -eq 0 ]; then
        curl -s -X 'GET' https://api.spotify.com/v1/tracks/$TRACK_ID -H
'Accept: application/json' -H 'Content-Type: application/json' -H
"Authorization: \"Bearer $myToken\"" | jq '.name, .artists[].name,
.album.name, .album.release_date, .track_number, .album.total_tracks' |
xargs printf "\"Music changed to '%s' from '%s' (album: '%s' in %s
(%s/%s))\"" | xargs notify-send --urgency=low --expire-time=3000 --
icon=/usr/share/icons/gnome/32x32/actions/player_fwd.png --app-
name=spotifyd spotifyd
    else
        echo "Cannot get token."
    fi
elif [ "$PLAYER_EVENT" = "stop" ];
then
    if [ $RESULT -eq 0 ]; then
        curl -s -X 'GET' https://api.spotify.com/v1/tracks/$TRACK_ID -H
'Accept: application/json' -H 'Content-Type: application/json' -H
"Authorization: \"Bearer $myToken\"" | jq '.name, .artists[].name,
.album.name, .album.release_date, .track_number, .album.total_tracks' |
```

```

xargs printf "Stoping music (Last song: '%s' from '%s' (album: '%s' in %s
(%s/%s)))\" | xargs notify-send --urgency=low --expire-time=3000 --
icon=/usr/share/icons/gnome/32x32/actions/player_stop.png --app-
name=spotifyd spotifyd
    else
        echo "Cannot get token."
    fi
else
    echo "Unknown event."
fi

```

- Make this script executable (`chmod +x notification_script.sh`)
- Add the line `onevent = "bash /home/YOU_USER/bin/spotifyNotifications.sh"` to your `spotifyd.conf`

Troubleshooting

no sound on FreeBSD

If you have correctly configured `spotifyd` to use `portaudio` and everything seems to be working except that there's no sound, you might have to switch to a different audio device.

If you have `portaudio` as your backend and set your device to `?`, `spotifyd` will output all of the available PortAudio devices it can find on your system. This could look like the following

- `/dev/dsp0` (default)
- `/dev/dsp1`
- `/dev/dsp2`
- `/dev/dsp`

Assume that the 4th device (index 3, starting from 0, `/dev/dsp`) is the output device that is needed. That coincides with the `pcm3` (also index 3, starting from 0) device that FreeBSD lists in `dmesg` as well as the `hw.snd.default_unit=3` `sysctl` that is used to set the device as OSS's default. It seems like the index number correlates across each of those enumerations.

After setting `device = "/dev/dsp"` in the config, the sound should start working. If not, you can try the other possible values that are available.